oLLaMa - zero, one & few shot

Introduction

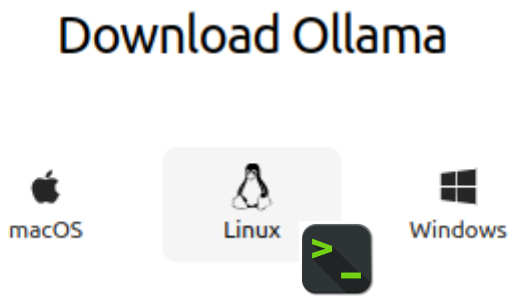
This short lab is designed to get you comfortable working with **oLLaMa**: we'll run some basic zero, one, and few-shot experiments using LLaMa3 or Gemma2. You are welcome to customize this lab by adding your comments, and observations about the model's output. You will find several assignments. The answers are located at the end of the lab.

Ollama offers a user-friendly interface for **running large language models (LLMs) locally**, particularly on MacOS, Linux and Windows platforms. This powerful tool provides immense value to researchers, developers, and anyone seeking to explore the capabilities of language models. Ollama supports a diverse range of models.

Ollama offers an easy installation and user experience, even for users unfamiliar with language models. Its simple API facilitates creating, running, and managing models effortlessly. Ollama utilizes GPU acceleration to speed up model inference by up to 2x compared to CPU-only setups. This benefit is particularly impactful for tasks requiring substantial computational power. This feature is readily available without any required intervention.

Running LLMs locally with Ollama ensures data privacy as your data remains on your device and never leaves your control. Additionally, it eliminates the need for costly inference fees, making it ideal for applications demanding a significant number of tokens.

Ready to get started? Download Ollama from: https://ollama.com/download



In short labs, we recommend utilizing either the terminal or a shell (a black screen displaying white or green characters). A shell acts as a command-line interpreter or a textual interface for executing commands and interacting with the operating system and LLMs using oLLaMa. However, you can also access oLLaMa through web interfaces. From this lab onwards, we will be utilizing the ChatGPT interface alongside traditional shell and graphical user interface approaches to explore both technologies comprehensively.

Attribution: Ander Barrena Madinabeitia

Basic Operations with Ollama

To find a model in Ollama, you can visit: https://ollama.com/library
This page showcases all available Ollama models for local deployment. Models are categorized by capability, with each model's page providing comprehensive details:

- Model Size: Indicates the file size of the model.
- Capabilities: Details the functionalities and specific abilities of the model.
- Usage Guide: Explains how to utilize the model (in most cases).

For our first shortLab, we will be using Llama3 or Gemma2. Please keep in mind that, as we are working with local model deployment, **this will require your computer's resources** (RAM, CPU, and potentially GPU).

With an 8B parameter LLaMa3 model, we require approximately 7~8GB of memory (32G in full precision). To address this constraint, we will utilize a quantized model. This is a process of compressing models by reducing the precision of their numerical values. Think of it like shrinking the size of a complex map while maintaining its essential details.

Benefits of Quantized LLMs:

- **Reduced Storage**: Saves space for applications, enabling deployment on consumer devices like phones and tablets.
- **Faster Performance**: Reduced computational load translates to quicker response times.
- **Lower Energy Consumption**: More energy efficient operation, making them more sustainable for long-term use.

Challenges of Quantization:

- **Trade-off with Accuracy**: There can be a slight decrease in accuracy as precision is reduced during quantization.

We recommend utilizing the **higher-quantized model** for faster performance llama3.1:8b-text-q4_K_S (7~8GB of RAM). However, if your computer is capable of handling non-quantized or even larger models, feel free to utilize those as well. If you continue to experience resource limitations check: gemma2:2b-text-q4_K_S (~2G of RAM and still achieving decent results 🤗) or llama3.2:1b-text-q8_0 (~1G of RAM).

If you continue to experience resource issues during the lab session, please ensure optimal use by closing unnecessary Chrome tabs and ending the video conference. Once finished, reconnect to the course.

Let's proceed. Open your terminal[1] and execute the following command (without $):

```
$ ollama run gemma2:2b-text-q4_K_S
```

Done! gemma2 is now ready for your prompts.[2]

```
$ >>> Send a message (/? for help)
```

ICL - Zero, one and few shot - translation

Try each prompt **multiple times** and to ensure the model operates independently of previous prompts, **use '/clear'** at the end of each command prompt. 👈 We will now attempt to recreate the experiments detailed in the GPT-3 paper (and the course slides) regarding zero-shot and few-shot learning for language translation by LLaMa3 or Gemma2.

> **Spoiler**: a quantized 2b SLM isn't as good as a 175b model, but it still reveals useful insights about acquired knowledge. The correct answer might appear within the generated text:

```
$ >>> Translate English to French:
cheese => fromage
meatball => merguez
Please help me with translating these two words from french to …
```

Zero shot (task description):

```
$ >>> Translate English to French:
cheese =>
```

One shot (task description + single example):

```
$ >>> Translate English to French:
sea otter => loutre de mer
cheese =>
```

Few shot (task description + **n** examples):

```
$ >>> Translate English to French:
sea otter => loutre de mer
peppermint => menthe poivrée
plush giraffe => girafe peluche
cheese =>
```

[1] For window users: use powerShell or cmd
[2] Check out the full details of different commands here: https://github.com/ollama/ollama

Tips

- To ensure the model operates independently of previous prompts, use '/clear' at the end of each command prompt. 👈

```
$ >>> /clear
```

- Or pass the prompt as an argument (This command isn't fully recommended because **line breaks are important** for this to work correctly)

```
$ ollama run llama3:8b-text-q4_K_S "Translate English to French: cheese =>"
```

- We suggest copying prompts from this notebook to ensure proper line breaks. While you can enter multi-line prompts by starting the multiline string with **"""**. Once you are done with the multiline input, close it with another **"""**. Be careful when copy-pasting triple quotes. If it doesn't work, enter the triple quotes manually using the keyboard; this will fix the problem.

```
$ >>> """Translate English to French:
… sea otter => loutre de mer
… peppermint => menthe poivrée
… plush giraffe => girafe peluche
… cheese =>"""
```

Assignments

1. Does Llama3 or Gemma2, accurately perform translations? Or do you encounter any inaccuracies in its output? **Sol1**
2. How does few-shot learning impact the accuracy of translation for this specific example? **Sol2**
3. Does LLaMa3 or Gemma2 always produce the same output for the same prompt, or is there variability in its responses? **Sol3**

Few shot - sentiment analysis[3]

We can observe that the model has somehow learned how to perform the task by providing it with just a few examples. For more difficult tasks, we can experiment with increasing the demonstrations.

```
$ >>> Perform sentiment analysis:
This is awesome! => Positive
This is bad! => Negative
Wow that movie was rad! => Negative
What a horrible show!  =>
```

Following the findings from Min et al. (2022), here are a few more tips about demonstrations/exemples when doing few-shot:

- The label space and the distribution of the input text specified by the demonstrations are both important (**regardless of whether the labels are correct for individual inputs**)
- **The format you use** also plays a key role in performance, even if you just use random labels, this is much better than no labels at all.
- Additional results show that **selecting random labels from a true distribution** of labels (instead of a uniform distribution) also helps.

Let's try out a few examples. Let's first try an example with random labels (meaning the labels Negative and Positive are randomly assigned to the inputs):

```
$ >>> Perform sentiment analysis:
This is awesome! => Negative
This is bad! => Positive
Wow that movie was rad! => Negative
What a horrible show!  =>
```

We still get the correct answer, even though the labels have been randomized. Note that we also kept the format, which helps too.

[3] edited from https://www.promptingguide.ai/techniques/fewshot

Zero Shot Question Answering

Now, let's see how our model performs with a Multiple Choice Q&A challenge! Instead of generating answers automatically, it will choose the correct response from several options provided (the answer is C):

```
$ >>> Respond always with a single letter A, B, C or D:
Question: How many kilograms are there in a tone?
A. 10,000 kilos
B. 1,000,000 kilos
C. 1,000 kilos
D. 100 kilos
Answer:
```

Now, formulate the question directly, allowing the model to provide the answer.

```
$ >>> Question: How many kilograms are there in a tone?
Answer:
```

Assignments

1. Does Llama3 or Gemma2 accurately respond to multiple-choice questions in a standardized format (e.g., A, B, C, D)? **Sol4**
2. Does the system give us correct answers when we let it generate them? **Sol5**

oLLama comands

We will briefly explore some Ollama commands that will help you customize both the framework and the models.

```
$ ollama
Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve       Start ollama
  create      Create a model from a Modelfile
  show        Show information for a model
  run         Run a model
  pull        Pull a model from a registry
  push        Push a model to a registry
  list        List models
  ps          List running models
  cp          Copy a model
  rm          Remove a model
  help        Help about any command

Flags:
  -h, --help       help for ollama
  -v, --version    Show version information

Use "ollama [command] --help" for more information about a command.
```

Using the ollama *list* command will display all the models available in the repository. You can remove unused models to free up disk space by using the ollama *rm* [model] command.

When running a model, you can type */?* for help and additional options.

```
$ ollama run gemma2:2b
$ >>> /?
Available Commands:
  /set            Set session variables
  /show           Show model information
  /load <model>   Load a session or model
  /save <model>   Save your current session
  /clear          Clear session context
  /bye            Exit
  /?, /help       Help for a command
  /? shortcuts    Help for keyboard shortcuts

Use """ to begin a multi-line message.
```

We've already seen the */clear* command, but you'll find the */set* command interesting as well. It allows you to customize various settings for the model during runtime.

```
$ ollama run gemma2:2b
$ >>> /set
Available Commands:
  /set parameter ...       Set a parameter
  /set system <string>     Set system message
  /set history             Enable history
  /set nohistory           Disable history
  /set wordwrap            Enable wordwrap
  /set nowordwrap          Disable wordwrap
  /set format json         Enable JSON mode
  /set noformat            Disable formatting
  /set verbose             Show LLM stats
  /set quiet               Disable LLM stats
```

Let's check the */set* command options. This command allows you to modify parameters such as temperature, max tokens, and more. Here's how it can be used:

```
$ ollama run gemma2:2b
$ >>> /set parameter
Available Parameters:
  /set parameter seed <int>                Random number seed
  /set parameter num_predict <int>         Max number of tokens to predict
  /set parameter top_k <int>               Pick from top k num of tokens
  /set parameter top_p <float>             Pick token based on sum of probabilities
  /set parameter min_p <float>             Pick token based on top token probability * min_p
  /set parameter num_ctx <int>             Set the context size
  /set parameter temperature <float>       Set creativity level
  /set parameter repeat_penalty <float>    How strongly to penalize repetitions
  /set parameter repeat_last_n <int>       Set how far back to look for repetitions
  /set parameter num_gpu <int>             The number of layers to send to the GPU
  /set parameter stop <string> <string> ...   Set the stop parameters
```

We are interested in:

- */set parameter temperature [value]*
  Adjusts the randomness of the model's responses. A lower value makes responses more deterministic, while a higher value increases diversity [0-1].
- */set parameter num_predict [value]*
  Limits the number of tokens in the model's output.

For example, you can adjust the previous two commands to make the model more deterministic and limit it to predicting only one word. After setting these parameters, try running the Question & Answering and Sentiment Analysis examples again, and check the output to see how the model behaves under these strict constraints.

All these options allow you to tune how the model behaves during interactions.
In future labs, we will revisit some commands and introduce new ones.

LLMs can handle some NLP tasks, as shown by these examples. However, we observed limitations in accuracy and tendencies towards parrot-like output, mimicking rather than true comprehension. Addressing these challenges has been achieved through techniques like

instruction tuning (IT) and **Reinforcement Learning from Human Feedback** (RLHF), which align LLMs with human communication for improved performance. This will be covered shortly 🙂

You are done? Review the GPT3 paper and attempt the NLP tasks along with their prompts.

Solutions

Note that the quantized 9b, 8b or 2b SLM isn't as good as a 175b model, but it still reveals useful insights about acquired knowledge. The correct answer might appear within the generated text. Solutions obtained by gemma2:9b-text-q4_K_S

Sol1 → While the translation is accurate, it produces more text than necessary. They can be quite repetitive.
Sol2 → In this instance, the translation is accurate and consistent across zero, one, and few-shot learning scenarios.
Sol3 → Randomness within a Probability Distribution: LLMs are trained on vast amounts of data. This means they learn statistical probabilities for generating words and phrases based on the surrounding context. However, each time they generate text, it's not precisely deterministic, there is randomness built-in.

Sol4 → It works correctly, however, the model tends to produce further text after completing the initial response.
Sol5 → No… They keep talking about weight and kilograms, however, it's important to note that we're working with small, quantized models. Scaling up these models will address this issue, although running them locally is not feasible, at least not on my personal computer.